

Search Typeahead System

A search-as-you-type suggestion engine with a distributed cache, recency-aware trending, and batched writes.

AUTHOR

Lakshya Bagani

DATE

22 June 2026

REPOSITORY

github.com/LakshyaBagani/TYPE-AHEAD

STACK

Node.js · Express · SQLite

1 Architecture diagram & explanation

2 Dataset source & loading instructions

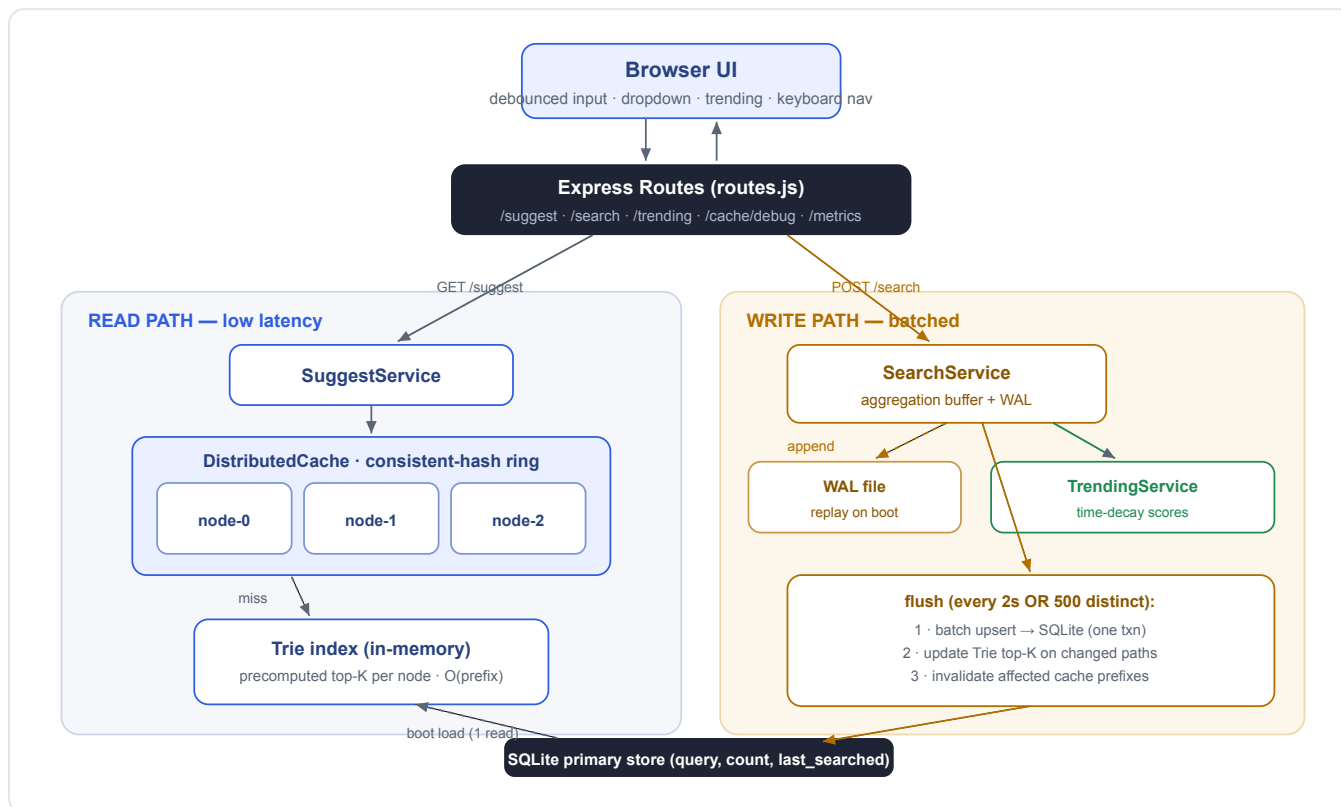
3 API documentation

4 Design choices & trade-offs

5 Performance report

1 Architecture

The system is organised as **three data layers** behind a thin HTTP layer. Each layer shields the one below it from traffic, so that after boot **per-request database reads are zero**.



Component responsibilities

- **SQLite (primary store)** — durable source of truth: `query`, `count`, `last_searched`.
- **Trie index (in-memory)** — prefix tree built from SQLite at boot, with a precomputed top-K list at every node. Serves prefix lookups in $O(\text{prefix length})$, no per-request scan.
- **Distributed cache** — N logical nodes; a consistent-hash ring routes each prefix key to one node and caches the *result list* with a TTL (plus LRU + invalidation).
- **Express HTTP layer** — the public endpoints listed in §3.
- **Batch writer + Trending** wrap the write path: a buffer/WAL coalesces writes, and an exponential time-decay model scores recency.

The two hot paths

```
READ (GET /suggest)
  prefix --normalize--> key "mode:prefix" --consistent hash--> cache node
  HIT -> return cached list (sub-microsecond)
  MISS -> Trie walk  $O(L)$  -> rank -> store in node (TTL) -> return

WRITE (POST /search)
  query --normalize--> append WAL --> aggregation buffer (+1) --> trending.record()
  --> return {"message":"Searched"} [no DB write on this path]
  flush (every 2s OR 500 distinct queries):
    one SQLite transaction (aggregated upserts)
    -> update Trie top-K on changed paths
    -> invalidate affected cache prefixes
    -> truncate WAL
```

Boot sequence & rationale

open SQLite → bulk-read all rows (1 read) → build Trie → create cache ring → replay WAL → start periodic flush → listen. The Trie shields the DB from *read* traffic; the cache shields the Trie from *repeated* reads; the batch writer shields the DB from *write* traffic. Each concern is a separate module behind a small interface, so any one (e.g. the in-process cache → real Redis) can be swapped without touching the others.

2 Dataset Source & Loading Instructions

Source

Wikimedia "pageviews" hourly dumps — <https://dumps.wikimedia.org/other/pageviews/> . Each line is `domain page_title view_count bytes` . We keep English (`en`) article titles as **queries** and their hourly **view counts** as popularity — real, openly-licensed data with genuine frequency values.

- Rows downloaded: **~216,506**; distinct queries after normalize/aggregate: **~215,890** (well above the 100k minimum).
- **Guarantee:** if the network or a dump file is unavailable, `download-dataset.js` tops up with realistic **synthetic** queries (Zipfian counts) so the size requirement can never fail the build.
- **Curated head:** a handful of queries (`iphone` , `iphone 15` , `java tutorial` , ...) are forced in with high counts so demos match the assignment examples.

Format

`query<TAB>count` per line in `data/queries.tsv` :

```
iphone      100000
iphone 15   85000
iphone charger 60000
java tutorial 40000
```

Loading

```
npm run dataset # download (or synthesize) -> data/queries.tsv
npm run ingest  # normalize + aggregate   -> data/typeahead.sqlite
npm run setup   # both of the above
```

Ingestion lowercases and whitespace-normalises each query with the **same** `normalize()` the API uses, then aggregates duplicates / case-variants into one row per distinct query (the rubric's "derive counts by aggregation"). Observed top rows after load:

```
100039 iphone
95193  google
90711  youtube
88530  chatgpt
85008  iphone 15
```

3 API Documentation

Base URL `http://localhost:3000` .

Method	Endpoint	Purpose
GET	<code>/suggest?q=<prefix>&mode=<basic enhanced></code>	Up to 10 prefix suggestions, ranked
POST	<code>/search</code>	Dummy search; records query (batched)
GET	<code>/trending</code>	Top-N by decayed recent score
GET	<code>/cache/debug?prefix=<p>&mode=<m></code>	Owning cache node + hit/miss for the key
GET	<code>/metrics</code>	Latency, hit rate, DB counts, write reduction, ring distribution
POST	<code>/admin/flush</code>	Force a batch flush (demo helper)
GET	<code>/health</code>	Liveness + row count

GET /suggest

Top-10 suggestions for a prefix. `q` is case-insensitive (empty/missing → empty list); `mode` is `basic` (all-time count) or `enhanced` (recency-aware blend, default). `cache` ∈ `hit` | `miss` | `skip` .

```
$ curl "localhost:3000/suggest?q=iphone&mode=basic"
{
  "query": "iphone", "mode": "basic", "cache": "miss",
  "suggestions": [
    { "query": "iphone", "count": 100039 },
    { "query": "iphone 15", "count": 85008 }
  ]
}
```

POST /search

Records the query (batched) and returns a fixed message.
Empty/missing query → 400 .

```
$ curl -X POST localhost:3000/search \
-H 'Content-Type: application/json' \
-d '{"query":"iphone 15"}'
{ "message": "Searched" }
```

GET /trending

Top-N queries right now, ranked by decayed recent score.

```
$ curl localhost:3000/trending
{ "trending": [
  { "query": "iphone 15", "score": 33.048 }
] }
```

GET /cache/debug

```
{ "prefix":"qx", "mode":"basic",
  "key":"basic:qx",
  "node":"cache-node-1", "hit":false }
```

GET /metrics (shape)

```
{ "latency":{"p50ms":0.06, "p95ms":0.34 },
  "cache":{"hitRate":0.98, "perNode":[...],
    "keyDistribution":{...} },
  "db":{"reads":1, "writeStatements":500 },
  "batch":{"writeReductionFactor":200 } }
```

4 Design Choices & Trade-offs

Decision	Why	Key trade-off
Node.js + Express	I/O-bound, low-latency; one language with the UI	Single-threaded — but that makes batch flush atomic, an advantage here
SQLite (better-sqlite3)	Zero-setup durable truth; sync API → exact write counts	Not multi-writer (fine: all writes funnel through one batcher)
Trie + precomputed top-K	O(prefix) lookups, pre-sorted results	Memory + slower writes → <i>motivates batching</i>
In-process logical cache nodes	"Logical" per rubric; easy to run; concept fully shown	Not fault-isolated like real Redis (interface lets us swap)
Consistent hashing + vnodes	Add/remove a node moves ~K/N keys, not all	Slightly more complex than modulo hashing
Exponential time-decay trending	O(1) memory/update; auto-forgets spikes	"Score" less interpretable than a windowed count; never exactly 0
Buffer + WAL batch writes	Aggregates duplicates; amortizes Trie updates; crash recovery	No per-write fsync → tiny loss window on power loss
Vanilla frontend	No build step; easiest to run	Less scalable than a framework for large UIs

Basic vs enhanced ranking (real output)

After searching `iphone 15` several times then flushing, `iphone 15` overtakes `iphone` in enhanced mode purely on recency, then decays back over time:

```
BASIC (all-time count), q=iph:
100039  iphone
85069   iphone 15
60000   iphone charger
52012   iphone 15 pro

ENHANCED (recency-aware), q=iph:
score=0.910  recent=31.97  iphone 15  <-- promoted
score=0.600  recent=0      iphone
score=0.360  recent=0      iphone charger
score=0.312  recent=0      iphone 15 pro
```

Consistent hashing (real output)

```
cache miss -> hit transition:      prefix routing spread across nodes:
/cache/debug qx -> node-1 hit:false  apple->node-1  banana->node-0  java->node-2
(GET /suggest qx)                 python->node-1  iphone->node-2  weather->node-0
/cache/debug qx -> node-1 hit:true   news->node-1   bitcoin->node-2
```

Each prefix deterministically maps to one node; keys spread across all three.

5 Performance Report

Measured with `npm run benchmark` against the real ~215.9k-query index (representative run on the dev machine).

98.1%
CACHE HIT RATE (ZIPFIAN, 100K REQ)

200×
FEWER DB WRITES VIA BATCHING

0
PER-REQUEST DB READS AFTER BOOT

5.2%
MAX RING DEVIATION (10K KEYS / 3 NODES)

5.1 Latency (p50 / p95 / p99)

Path	p50	p95	p99
Cache hit	0.0008 ms	0.0012 ms	0.0038 ms
Cache miss (Trie lookup)	0.0127 ms	0.021 ms	0.044 ms

Both paths are well under a millisecond; the hit path is ~15× faster than a Trie miss, and the miss path never touches SQLite.

5.2 Cache hit rate

Zipfian workload, 100,000 requests over ~1,857 distinct prefixes:
98.1% hit rate, mixed p95 = 0.0015 ms. Per-node load (hits): node-0 ≈ 40.4k, node-1 ≈ 30.2k, node-2 ≈ 29.4k.

5.3 DB read/write counts

Per-request DB reads after boot: **0** — served entirely from Trie + cache. SQLite is read exactly **once** at boot. Writes happen only on flush, one statement per distinct query per flush.

5.4 Write reduction (batching)

100,000 `POST /search` calls drawn from 500 distinct queries → **500** DB write statements in one transaction = **200×** reduction. (Live check: 81 searches → 3 writes = 27× with fewer distinct queries.)

5.5 Consistent-hash distribution

10,000 keys over 3 nodes (150 vnodes each): node-0 = 3,508, node-1 = 3,239, node-2 = 3,253. Ideal = 3,333/node → **max deviation 5.2%**.

5.6 Index build

Trie built from ~215.9k queries in **~1.6–1.7 s** at boot (one-time cost).

Summary. The design meets the low-latency goal (sub-ms p95 on both paths), keeps the cache hit rate high under realistic skew, balances keys evenly across nodes, and cuts database writes by two orders of magnitude under repeated-query load — while keeping per-request DB reads at zero.